

MT5 Futures Hedging Dashboard

Technical Reference & Architecture Book

A deep-dive into the Flask backend, SQLite data layer, single-page JavaScript frontend, MetaTrader 5 desktop connector, and production deployment strategy powering a multi-tier proprietary trading management platform.

Version 1.2.0 · March 01, 2026
github.com/Oukaharry/FuturesAutomatedFeed
Author: Oukaharry · Branch: main

TABLE OF CONTENTS

TABLE OF CONTENTS	3
CHAPTER 1: SYSTEM OVERVIEW	8
1.1 What This System Does	8
1.2 Architecture Diagram	8
1.3 User Roles & Permissions	8
1.4 Data Flow	8
CHAPTER 2: FLASK BACKEND	10
2.1 Application Bootstrap	10
2.2 Authentication Decorators	10
2.3 Key API Endpoints	10
2.4 /api/get_data — Data Retrieval	11
2.5 /api/update_data — Dual-Auth Merge	11
2.6 Route Structure	12
2.7 Account Signature Matching	12
CHAPTER 3: DATABASE LAYER	13
3.1 Engine & Connection Strategy	13
3.2 Schema Reference	13
3.3 Password Security	14
3.4 Login Verification Flow	14
3.5 Versioned History & Rollback	14
3.6 Soft Delete Pattern	14
CHAPTER 4: FRONTEND APPLICATION	15
4.1 Architecture Choice	15
4.2 Layout Structure	15
4.3 Sticky Header System	15
4.4 Scroll Section Indicator (Pill Badge)	15

4.5	Pagination Strategy	16
4.6	Inline Editing & Save Flow	16
4.7	Brand Colour System	16
CHAPTER 5: TRADER COMPANION DESKTOP APP		17
5.1	Purpose	17
5.2	MT5 Data Extraction	17
5.3	Evaluation Matching Algorithm	17
5.4	Push Protocol	17
5.5	Desktop Build (PyInstaller)	17
CHAPTER 6: FINANCIAL CALCULATIONS MODULE		19
6.1	Overview	19
6.2	Core Functions	19
6.3	Watermark Scheduler	19
CHAPTER 7: DEPLOYMENT & OPERATIONS		20
7.1	Production Stack	20
7.2	Gunicorn Configuration	20
7.3	Nginx Reverse Proxy (sample)	20
7.4	Environment Variables	20
7.5	Deployment Script	20
7.6	Rate Limiting Strategy	21
CHAPTER 8: SECURITY ARCHITECTURE		22
8.1	Defence in Depth	22
8.2	Brute-Force Protection	22
8.3	SQL Injection Prevention	22
8.4	Session Management	22
8.5	Audit Log	22
CHAPTER 9: DEVELOPER WORKFLOWS		23
9.1	Local Development Setup	23

9.2 Common Debug Scripts	23
9.3 Git Workflow	23
9.4 Building the Desktop App	23
CHAPTER 10: REST API REFERENCE	24
10.1 Authentication	24
10.2 POST /api/auth/login	24
10.3 GET /api/get_data	24
10.4 POST /api/update_data	24
10.5 POST /api/rollback	24
10.6 Error Responses	25
CHAPTER 11: COMPLETE FILE INVENTORY	26
11.1 Backend — Core Modules	26
11.2 Backend — Utility Modules	26
11.3 Config Modules	26
11.4 Desktop Application	27
11.5 Frontend Templates & Static Assets	27
11.6 Build & Deployment Scripts	27
11.7 Debugging & Utility Scripts	27
CHAPTER 12: APP.PY — COMPLETE FUNCTION REFERENCE	29
12.1 Helper & Utility Functions	29
12.2 Auth Decorators	29
12.3 Web (HTML) Routes	30
12.4 Authentication API Routes	30
12.5 User Management API Routes	31
12.6 Data API Routes	31
12.7 Version History & Rollback Routes	32
12.8 API Key Management Routes	32
12.9 Trader Push Routes (Granular)	33

12.10 System / Stats Routes	33
---------------------------------------	----

CHAPTER 13: DATABASE.PY — COMPLETE FUNCTION REFERENCE 34

13.1 Connection & Initialisation	34
13.2 Password & Admin	34
13.3 User CRUD	34
13.4 Brute-Force Protection	35
13.5 API Key Management	35
13.6 Client Data Storage	35
13.7 Version History	36
13.8 Audit Log	36
13.9 Session Management	36
13.10 Migration Utility	37

CHAPTER 14: FINANCIAL_OVERVIEW.PY — FUNCTION REFERENCE 38

14.1 Infrastructure / Cache	38
14.2 Data Normalisation Helpers	38
14.3 Payout & Growth Functions	38
14.4 MT5 Deal Data Functions	38
14.5 Overhead & Hedge Functions	39
14.6 Aggregate Overview Functions	39
14.7 Data Access Helper	39

CHAPTER 15: TRADER COMPANION — FULL FUNCTION REFERENCE 40

15.1 MT5DataPusher Class	40
15.2 TraderCompanionApp Class	41
15.3 Module-Level Functions	42

CHAPTER 16: SUPPORTING MODULES — FUNCTION REFERENCE 43

16.1 dashboard/scheduler.py	43
16.2 dashboard/notes_service.py	43
16.3 dashboard/watermark_service.py	43

16.4 dashboard/utils/trade_matcher.py — UnifiedTradeMatcher	44
16.5 dashboard/phase_manager.py — PhaseManager	44
16.6 config/hierarchy.py	45

CHAPTER 17: JAVASCRIPT FUNCTIONS — INDEX.HTML REFERENCE 46

17.1 Initialisation & Data Loading	46
17.2 Evaluations Table	46
17.3 Cell Editing & Input Helpers	46
17.4 Inline Notes System	47
17.5 Scroll & Section Navigation	47
17.6 Stats, Positions & Accounts	47
17.7 Trade History Tab	48
17.8 Hedging Review Tab	48
17.9 Save, Version History & Rollback	49
17.10 Waterlog / Watermark Tab	49

CHAPTER 1: SYSTEM OVERVIEW

1.1 What This System Does

The **MT5 Futures Hedging Dashboard** is a full-stack web application that aggregates, tracks, and visualises the proprietary trading activities of multiple traders across several prop-firm evaluation pipelines. It pulls live data from MetaTrader 5 trading accounts, stores it in a structured database, and exposes that data through a role-based web dashboard and a secured REST API.

The system supports a four-tier hierarchy: **Super Admin** → **Admin** → **Trader** → **Client**. Each tier has progressively scoped access rights enforced at both the route level (via decorator guards) and the database query level.

1.2 Architecture Diagram

Layer	Technology	Role
Desktop App	Python + MetaTrader5 API	Extracts live MT5 data, pushes to dashboard via API
Web Backend	Flask 3 + Gunicorn	REST API, session management, role-based access control
Database	SQLite (dev) / PostgreSQL (prod)	Persistent storage of client data, audit log, history
Frontend	Vanilla JS + CSS3	Single-page dashboard, inline editing, infinite scroll
Auth	PBKDF2-SHA256 sessions + API keys	Cookie sessions for UI; X-API-Key header for apps
Deployment	Ubuntu VPS + Nginx + Gunicorn	Reverse-proxy, SSL termination, process management

1.3 User Roles & Permissions

Role	Login Method	Can See	Can Edit
super_admin	Password (PBKDF2)	All admins, traders, clients	Everything
admin	Password (PBKDF2)	Own traders + clients	Trader/client data
trader	Password (PBKDF2)	Own clients only	Client evaluations
client	Email + Password	Own data only	Prop Day notes only
API (Trader App)	X-API-Key header	Scoped to trader's clients	Push MT5 data

1.4 Data Flow

- The **Trader Companion** desktop app connects to a live MT5 terminal and extracts deals, positions, account balances, and evaluation states at a configurable interval (typically every 60 seconds).
- The data is serialised to JSON and POSTed to `/api/update_data` on the Flask server using an **X-API-Key** header for authentication.
- The server merges the incoming payload with existing client data, preserving soft-deleted evaluation rows and dashboard-side edits, then saves to the database and writes a versioned snapshot to the

audit/history tables.

4. The web dashboard polls `/api/get_data?client_id=...` and renders the data into a rich HTML table with sticky headers, inline editing, and an expandable financials panel.

CHAPTER 2: FLASK BACKEND

2.1 Application Bootstrap

The Flask application lives in **dashboard/app.py** (~4 400 lines). On startup it initialises the database, starts the midnight watermark scheduler, configures rate limiting via **Flask-Limiter**, and sets a cryptographically random secret key for sessions.

■ dashboard/app.py – app initialisation

```
app = Flask(__name__) app.config['SEND_FILE_MAX_AGE_DEFAULT'] = 0 app.secret_key =
os.getenv('FLASK_SECRET_KEY', secrets.token_hex(32)) limiter = Limiter( app=app,
key_func=get_remote_address, default_limits=["10000 per day", "2000 per hour"],
storage_uri="memory://" ) # Start Midnight Watermark Scheduler from dashboard.scheduler import
start_scheduler start_scheduler()
```

2.2 Authentication Decorators

Four decorators guard every route. They are composed using Python's `functools.wraps` to preserve the original function metadata.

require_api_key

Extracts the X-API-Key header, validates it against the database, injects the resolved `request.api_user` context, and logs the access.

■ dashboard/app.py – require_api_key decorator

```
def require_api_key(f): @wraps(f) def decorated_function(*args, **kwargs): api_key =
request.headers.get('X-API-Key') client_ip = get_remote_address() if not api_key:
log_action('API_ACCESS_DENIED', 'unknown', 'no_key', client_ip, 'Missing API key', False)
return jsonify({"status": "error", "message": "API key required"}), 401 user_info =
validate_api_key(api_key) if not user_info: log_action('API_ACCESS_DENIED', 'unknown',
api_key[:12], client_ip, 'Invalid API key', False) return jsonify({"status": "error",
"message": "Invalid API key"}), 403 request.api_user = user_info log_action('API_ACCESS',
'trader', user_info.get('trader', 'unknown'), client_ip, f"Endpoint: {request.endpoint}")
return f(*args, **kwargs) return decorated_function
```

require_role(*allowed_roles)

Session-cookie based role gate. Reads the `session_token` cookie, validates it, and checks `user_type` against a whitelist of allowed roles. Returns HTTP 403 on mismatch.

■ dashboard/app.py – require_role decorator

```
def require_role(*allowed_roles): def decorator(f): @wraps(f) def decorated_function(*args,
**kwargs): session_token = request.cookies.get('session_token') if not session_token: return
jsonify({"status": "error", "message": "Authentication required"}), 401 session_info =
validate_session(session_token) if not session_info: return jsonify({"status": "error",
"message": "Invalid or expired session"}), 401 user_type = session_info.get('user_type') if
user_type not in allowed_roles: log_action('ACCESS_DENIED', user_type,
session_info.get('user_identifier'), get_remote_address(), f"Required roles: {allowed_roles}",
False) return jsonify({"status": "error", "message": "Access denied."}), 403
request.session_user = session_info return f(*args, **kwargs) return decorated_function return
decorator
```

2.3 Key API Endpoints

Method	Endpoint	Auth	Description
GET	/api/get_data	Session / API Key	Fetch client JSON data
POST	/api/update_data	Session / API Key	Push new MT5 snapshot
POST	/api/login	None (public)	Username+password login, returns session cookie
POST	/api/auth/login	None (public)	Unified multi-role login endpoint
POST	/api/rollback	Session	Rollback client data to a previous version
POST	/api/notes	Session	Save / delete inline cell notes
GET	/api/history	Session	List version history for a client
POST	/api/generate_key	Admin Password	Generate a new API key for a trader
GET	/api/audit_log	Session (admin+)	Return recent audit log entries

2.4 /api/get_data — Data Retrieval

This endpoint serves as the primary data source for the dashboard. It accepts both session cookies (for browser clients) and API-key headers (for the trader companion app), resolves the caller's identity, checks access permissions with `can_access_client()`, and enriches the raw JSON with inline notes and computed historical MT5 totals before returning the response.

■ dashboard/app.py – get_data() (condensed)

```
@app.route('/api/get_data') def get_data(): session_token =
request.cookies.get('session_token') api_key = request.headers.get('X-API-Key') # --- resolve
caller identity --- if session_token: session_info = validate_session(session_token) user_type
= session_info['user_type'] user_identifier = session_info['user_identifier'] elif api_key:
key_info = validate_api_key(api_key) user_type = 'api' user_identifier = key_info['owner']
else: return jsonify({"status": "error", "message": "Auth required"}), 401 client_id =
request.args.get('client_id') if not can_access_client(user_type, user_identifier, client_id):
return jsonify({"status": "error", "message": "Access denied"}), 403 data =
get_client_data(client_id) # Inject visual notes notes = get_client_notes(client_id) for i, ev
in enumerate(data.get('evaluations', [])): if i in notes: ev['_notes'] = notes[i]
data['status'] = 'success' return jsonify(data)
```

2.5 /api/update_data — Dual-Auth Merge

The update endpoint performs a server-side merge strategy: incoming data is layered on top of existing data so that missing keys (e.g. the dashboard user did not send deals) default to their current stored values. A versioned snapshot is written to the history table after every successful save.

■ dashboard/app.py – update_data() core merge logic

```
@app.route('/api/update_data', methods=['POST']) @limiter.limit("60 per minute") def
update_data(): data = request.json existing_data = get_client_data(client_id) or {} evaluations
= normalize_evaluations( data.get('evaluations', existing_data.get('evaluations', [])) )
client_data = { 'deals': data.get('deals', existing_data.get('deals', [])), 'positions':
data.get('positions', existing_data.get('positions', [])), 'account': data.get('account',
existing_data.get('account', {})), 'evaluations': evaluations, 'statistics':
data.get('statistics', existing_data.get('statistics', {})), # ... other fields merged
similarly } success, version = save_client_data_with_history( client_id, client_data,
action='UPDATE', changed_by=user_identifier, changed_by_type=user_type,
ip_address=get_remote_address(), change_source='dashboard_edit', change_description=f'Manual
edit by {user_type}' ) return jsonify({"status": "success", "version": version})
```

2.6 Route Structure

Routes are organised into logical groups. All routes serving HTML pages are decorated with `@require_session` for redirect-to-login behaviour, while JSON API routes return structured error objects.

Route Pattern	Template	Access
/	login.html	Public
/super_admin	super_admin.html	super_admin only
/admin/	admin_dashboard.html	admin / super_admin
/trader/	index.html	trader / admin / super_admin
/dashboard/	index.html	client (own data only)
/financial_overview	financial_overview.html	super_admin only

2.7 Account Signature Matching

MT5 deal comments often contain truncated account numbers (e.g. FNFT...59574). The server resolves these to evaluation rows using a signature-based algorithm that extracts the first-4 and last-4/5 digits normalised to lowercase.

■ `dashboard/app.py – get_account_signature()`

```
def get_account_signature(account_number): account_str = str(account_number).strip() #
Truncated format: PREFIX...SUFFIX e.g. FNFT...59574 if '...' in account_str: parts =
account_str.split('...') prefix = parts[0][:4] if len(parts[0]) >= 4 else parts[0] suffix =
parts[1] # keep full suffix return (prefix + suffix).lower() # Standard: first 4 + last 4 if
len(account_str) < 8: return account_str.lower() return (account_str[:4] +
account_str[-4:]).lower()
```

CHAPTER 3: DATABASE LAYER

3.1 Engine & Connection Strategy

The database module (**dashboard/database.py**, ~1 300 lines) uses Python's standard sqlite3 library in development, with a thin abstraction layer that can be swapped for psycopg2 (PostgreSQL) in production by changing a single environment variable.

All connections are obtained via a context manager (`get_connection()`) that sets `row_factory = sqlite3.Row` for named-column access and enables WAL journal mode for better concurrent read performance.

3.2 Schema Reference

user_credentials

■ **dashboard/database.py – user_credentials table DDL**

```
CREATE TABLE IF NOT EXISTS user_credentials ( id INTEGER PRIMARY KEY AUTOINCREMENT, username
TEXT NOT NULL, email TEXT, password_hash TEXT NOT NULL, salt TEXT NOT NULL, user_type TEXT NOT
NULL, -- super_admin | admin | trader | client parent_admin TEXT, -- hierarchy link
parent_trader TEXT, -- hierarchy link is_active INTEGER DEFAULT 1, must_change_password INTEGER
DEFAULT 0, last_login TEXT, created_at TEXT DEFAULT (datetime('now')), updated_at TEXT DEFAULT
(datetime('now')), UNIQUE(username, user_type) )
```

clients_data

■ **dashboard/database.py – clients_data table DDL**

```
CREATE TABLE IF NOT EXISTS clients_data ( id INTEGER PRIMARY KEY AUTOINCREMENT, client_id TEXT
NOT NULL UNIQUE, deals TEXT DEFAULT '[]', -- JSON array positions TEXT DEFAULT '[]', -- JSON
array account TEXT DEFAULT '{}', -- JSON object evaluations TEXT DEFAULT '[]', -- JSON array
(main eval sheet rows) updated_at TEXT DEFAULT (datetime('now')) )
```

audit_log

■ **dashboard/database.py – audit_log table DDL**

```
CREATE TABLE IF NOT EXISTS audit_log ( id INTEGER PRIMARY KEY AUTOINCREMENT, timestamp TEXT NOT
NULL, action TEXT NOT NULL, -- e.g. 'DATA_UPDATE', 'LOGIN', 'ROLLBACK' user_type TEXT,
user_identifier TEXT, ip_address TEXT, description TEXT, success INTEGER DEFAULT 1, client_id
TEXT )
```

data_history (version snapshots)

■ **dashboard/database.py – data_history table DDL**

```
CREATE TABLE IF NOT EXISTS data_history ( id INTEGER PRIMARY KEY AUTOINCREMENT, client_id TEXT
NOT NULL, version INTEGER NOT NULL, data_snapshot TEXT NOT NULL, -- full JSON snapshot action
TEXT, changed_by TEXT, changed_by_type TEXT, ip_address TEXT, change_source TEXT,
change_description TEXT, created_at TEXT DEFAULT (datetime('now')) )
```

daily_watermarks & waterlog_periods

■ **dashboard/database.py – watermark tables DDL**

```
CREATE TABLE IF NOT EXISTS daily_watermarks ( id INTEGER PRIMARY KEY AUTOINCREMENT, client_id
TEXT NOT NULL, eval_index INTEGER NOT NULL, date TEXT NOT NULL, balance REAL, drawdown_pct REAL,
created_at TEXT DEFAULT (datetime('now')), UNIQUE(client_id, eval_index, date) ); CREATE TABLE
IF NOT EXISTS waterlog_periods ( id INTEGER PRIMARY KEY AUTOINCREMENT, client_id TEXT NOT NULL,
eval_index INTEGER NOT NULL, start_date TEXT NOT NULL, end_date TEXT, peak_balance REAL,
```

```
trough_balance REAL, max_drawdown_pct REAL, status TEXT DEFAULT 'open' );
```

3.3 Password Security

Passwords are hashed using PBKDF2-HMAC-SHA256 with 100 000 iterations and a cryptographically random 32-byte salt generated by `secrets.token_hex(32)`. Comparison uses `secrets.compare_digest()` to prevent timing attacks.

■ [dashboard/database.py](#) – `hash_password` / `verify_password`

```
import secrets, hashlib def hash_password(password: str, salt: str = None) -> tuple: if salt is None: salt = secrets.token_hex(32) # 64-char hex string password_hash = hashlib.pbkdf2_hmac('sha256', password.encode('utf-8'), salt.encode('utf-8'), 100_000 # OWASP-recommended iteration count ).hex() return password_hash, salt def verify_password(password: str, stored_hash: str, salt: str) -> bool: computed_hash, _ = hash_password(password, salt) return secrets.compare_digest(computed_hash, stored_hash) # constant-time
```

3.4 Login Verification Flow

■ [dashboard/database.py](#) – `verify_user_password()` (condensed)

```
def verify_user_password(username, password, user_type): with get_connection() as conn: cursor = conn.cursor() cursor.execute(''' SELECT id, username, email, password_hash, salt, user_type, parent_admin, parent_trader, must_change_password FROM user_credentials WHERE username = ? AND user_type = ? AND is_active = 1 ''', (username, user_type)) row = cursor.fetchone() if row is None: return None # user not found if not verify_password(password, row['password_hash'], row['salt']): return None # wrong password # Stamp last_login cursor.execute( 'UPDATE user_credentials SET last_login = ? WHERE id = ?', (datetime.now().isoformat(), row['id']) ) conn.commit() return { 'id': row['id'], 'username': row['username'], 'email': row['email'], 'user_type': row['user_type'], 'parent_admin': row['parent_admin'], 'parent_trader': row['parent_trader'], 'must_change_password': bool(row['must_change_password']) }
```

3.5 Versioned History & Rollback

Every data mutation triggers `save_client_data_with_history()`, which writes a full JSON snapshot to `data_history` with an incrementing version counter. The dashboard's version history panel lists these snapshots, and any version can be restored via `/api/rollback`.

■ Rollback is non-destructive: the restored snapshot itself becomes version N+1, so the full history is always preserved and any rollback can itself be rolled back.

3.6 Soft Delete Pattern

Evaluation rows deleted from the dashboard are marked with a `_deleted: true` flag in the JSON array rather than being physically removed. This means concurrent pushes from the Trader Companion app cannot accidentally resurrect deleted rows.

■ [dashboard/templates/index.html](#) – `soft delete (JS)`

```
// Mark as soft-deleted instead of splicing the array currentData.evaluations[index]._deleted = true; // Render loop skips deleted rows rowsToRender = currentData.evaluations.filter( ev => !ev._deleted );
```

CHAPTER 4: FRONTEND APPLICATION

4.1 Architecture Choice

The entire UI is delivered as a single HTML file — **dashboard/templates/index.html** (~4 300 lines). No JavaScript framework or build tool is used; all interactivity is implemented in vanilla ES6+ code embedded directly in the template. This choice eliminates build complexity and makes the app trivially deployable as a Jinja2 template served by Flask.

4.2 Layout Structure

DOM Element	CSS Class / ID	Role
	.header-title-bar + .header-controls-bar	Sticky two-row header: logo/title + tabs/controls
	#main-content	Scrollable content area below the header
	#eval-table	Main evaluations data grid (~60+ columns)
	—	Three sticky header rows: group labels, column names, filters
	#eval-tbody	Rendered evaluation rows (paginated, 50/page)
	#eval-load-more-bar	'Show More' pagination trigger bar
	#section-sticky-label	Floating pill inside EVAL INFO header showing current scroll section

4.3 Sticky Header System

The table uses a three-tier sticky-header system implemented entirely in CSS. Each tier is positioned with a different top offset so all three rows remain visible when the user scrolls down through rows.

■ [dashboard/static/css/style.css – sticky header tiers](#)

```
/* Row 1: Section group labels (EVAL INFO, EVAL PHASE, etc.) */ thead tr:nth-child(1) th {
position: sticky; top: 0; z-index: 3000; } /* Row 2: Column names */ thead tr:nth-child(2) th {
position: sticky; top: 42px; /* height of row 1 */ z-index: 2000; } /* Row 3: Column filter
inputs */ thead tr:nth-child(3) th { position: sticky; top: 84px; /* row 1 + row 2 heights */
z-index: 1500; } /* Full-page header also sticky */ header { position: sticky; top: 0; z-index:
5000; flex-direction: column; }
```

4.4 Scroll Section Indicator (Pill Badge)

A coloured pill badge inside the EVAL INFO group header updates in real-time as the user scrolls the table horizontally, showing which logical section is currently in view (e.g. EVAL INFO, EVAL PHASE, FUNDED PHASE, FARMING).

■ [dashboard/templates/index.html – updateScrollIndicator\(\)](#)

```
const sectionLabelMap = { 'eval-info': { label: 'EVAL INFO', color: '#60a5fa' }, // blue
'eval-phase': { label: 'EVAL PHASE', color: '#4ade80' }, // green 'funded-phase':{ label:
'FUNDED PHASE',color: '#fb7185' }, // pink-red 'farming': { label: 'FARMING', color: '#c084fc'
}, // purple }; function updateScrollIndicator() { const pill =
document.getElementById('section-sticky-label'); if (!pill) return; const tableRect =
evalTable.getBoundingClientRect(); const viewCenter = tableRect.left + tableRect.width / 2; //
```

```
Find which group header overlaps the centre of the viewport for (const [key, config] of
Object.entries(sectionLabelMap)) { const header =
document.querySelector(`th[data-section="${key}"]`); if (!header) continue; const r =
header.getBoundingClientRect(); if (r.left <= viewCenter && r.right >= viewCenter) {
pill.textContent = config.label; pill.style.background = config.color + '33'; // 20% opacity
fill pill.style.color = config.color; pill.style.borderColor= config.color; break; } } //
Attach to both scroll axes evalTable.closest('.table-wrapper') .addEventListener('scroll',
updateScrollIndicator, { passive: true });
```

4.5 Pagination Strategy

Evaluation rows are rendered in pages of 50 to avoid DOM congestion on accounts with hundreds of evaluations. A sticky 'Show More' bar at the bottom of the table loads the next batch without any network request — all data is already resident in the JavaScript `currentData` object.

■ [dashboard/templates/index.html – pagination globals](#)

```
let evalPage = 1; const EVAL_PAGE_SIZE = 50; function renderEvaluationsTable(data, append =
false) { if (!append) evalPage = 1; const all = (data.evaluations || []).filter(ev =>
!ev._deleted).sort((a, b) => b.originalIndex - a.originalIndex); // newest first const slice =
all.slice(0, evalPage * EVAL_PAGE_SIZE); // ... build and insert table rows ... // Show / hide
the 'load more' bar const bar = document.getElementById('eval-load-more-bar'); const more =
all.length > slice.length; bar.style.display = more ? 'flex' : 'none'; if (more) {
bar.querySelector('.load-more-count').textContent = `${all.length - slice.length} more`; } }
function loadMoreEvaluations() { evalPage++; renderEvaluationsTable(currentData, true); }
```

4.6 Inline Editing & Save Flow

Each cell that can be edited triggers a `saveData()` call which PATCHes the entire current state back to the server. The function throttles saves to avoid hammering the API on rapid keystrokes using a debounce timer.

■ [dashboard/templates/index.html – saveData\(\) \(condensed\)](#)

```
let saveTimer = null; function saveData(changeDescription = 'Cell edit') {
clearTimeout(saveTimer); saveTimer = setTimeout(async () => { const payload = { ...currentData,
client_id: currentClientId, action_type: 'UPDATE', change_description: changeDescription };
const resp = await fetch('/api/update_data', { method: 'POST', headers: { 'Content-Type':
'application/json' }, body: JSON.stringify(payload) }); const result = await resp.json(); if
(result.status === 'success') { showToast('Saved ✓', 'success'); currentVersion =
result.version; } else { showToast('Save failed: ' + result.message, 'error'); } }, 600); // 600
ms debounce }
```

4.7 Brand Colour System

Each prop firm has a distinct brand accent colour applied to evaluation rows and UI indicators, making it instantly clear which firm is associated with each account row.

Prop Firm	Brand Colour	Hex	Applied To
Topstep	Yellow / Gold	#fcd34d	Row accent, badges, indicators
Lucid	Black / Slate	#334155	Row accent, muted tones
Alpha	Green	#4ade80	Row accent, profit indicators
FTMO	Blue	#60a5fa	Row accent
My Forex Funds	Orange	#fb923c	Row accent

CHAPTER 5: TRADER COMPANION DESKTOP APP

5.1 Purpose

The **Trader Companion** is a Windows-native Python application built with Tkinter that runs continuously alongside a MetaTrader 5 terminal. It uses the official MetaTrader5 Python package to fetch live trading data and push it to the dashboard server every 60 seconds.

5.2 MT5 Data Extraction

■ `trader_companion/trader_app.py – live data fetch (pattern)`

```
import MetaTrader5 as mt5 def fetch_mt5_data(self): if not mt5.initialize(): self.log('MT5 init failed') return None # Account summary account_info = mt5.account_info()._asdict() # All historical deals (closed trades) deals = mt5.history_deals_get( datetime(2020, 1, 1), datetime.now() ) deals_list = [d._asdict() for d in deals] if deals else [] # Open positions positions = mt5.positions_get() positions_list = [p._asdict() for p in positions] if positions else [] mt5.shutdown() return { 'account': account_info, 'deals': deals_list, 'positions': positions_list }
```

5.3 Evaluation Matching Algorithm

The most complex part of the Trader Companion is matching MT5 deal comments (which contain truncated account numbers) to rows in the user's evaluation spreadsheet. The matching uses a multi-strategy fallback: exact match → signature match (first4+last4 digits) → last-N-digits match.

Strategy	Input	Match Condition
1. Exact	Full account string	<code>account_str == eval['Account #']</code>
2. Signature	First4 + Last4 digits	<code>signature(deal) == signature(eval['Account #'])</code>
3. Last-N	Last 5 digits	<code>deal[-5:] == eval['Account #'][-5:]</code>
4. Prefix	First 4 chars	Used only for firm-specific disambiguation

5.4 Push Protocol

After processing, the app sends a POST request to `/api/update_data` with the full payload including deals, positions, and updated evaluations. Regular pushes send evaluations: `[]`, preserving server-side edits. Hedging sync pushes first fetch the current evaluations, update the hedge-related fields, then push the merged result.

■ `trader_companion/trader_app.py – push call (pattern)`

```
def push_to_server(self, data: dict, include_evaluations: bool = False): if not include_evaluations: data['evaluations'] = [] # preserve server-side eval edits response = requests.post( f"{self.server_url}/api/update_data", json=data, headers={'X-API-Key': self.api_key}, timeout=30 ) result = response.json() if result.get('status') == 'success': self.log(f"Pushed. Server version: {result.get('version')}") else: self.log(f"Push failed: {result.get('message')}")
```

5.5 Desktop Build (PyInstaller)

The app is distributed as a standalone .exe using PyInstaller. The spec file bundles all Python dependencies, the Tkinter runtime, and any required assets into a single-file executable that end users can run on Windows without installing Python or any packages.

Spec File	Version	Entry Point
Trader_Companion_v1.2.0.spec	1.2.0	trader_companion/trader_app.py
BallerQuotes_Trader_Companion_v1.2.0.spec	1.2.0	trader_companion/baller_app.py

■ Build command

```
# From workspace root, with PyInstaller installed: pyinstaller Trader_Companion_v1.2.0.spec
--clean # Output: dist/Trader_Companion_v1.2.0/Trader_Companion_v1.2.0.exe
```

CHAPTER 6: FINANCIAL CALCULATIONS

MODULE

6.1 Overview

`dashboard/financial_overview.py` is the analytics engine. It processes raw deal/position data from the database and produces aggregated financial metrics: running P&L, deposit growth, fee curves, hedge efficiency, and per-trader performance statistics.

6.2 Core Functions

Function	Output
<code>calculate_propfirm_overview()</code>	Aggregated P&L, deposits, withdrawals per prop firm
<code>get_payouts_history()</code>	Chronological list of payout events
<code>get_portfolio_growth_data()</code>	Time-series of combined portfolio value
<code>get_cumulative_deposits()</code>	Running deposit total over time
<code>get_cumulative_trading_profit()</code>	Running realised profit over time
<code>get_cumulative_fees_data()</code>	Running fee expenditure over time
<code>get_cumulative_hedge_data()</code>	Running hedge result vs. sheet value
<code>calculate_trader_stats()</code>	Per-trader win rate, avg profit, drawdown stats
<code>get_client_performance_stats()</code>	Per-client performance breakdown
<code>get_cached_clients_dataset()</code>	Memoised full dataset across all clients

6.3 Watermark Scheduler

A background thread (`dashboard/scheduler.py`) wakes at midnight UTC every day, iterates over all active evaluations, and records a **daily watermark** — the end-of-day balance and maximum drawdown — to the `daily_watermarks` table. This enables accurate drawdown tracking across multi-day sessions.

■ Phase transitions (e.g. Challenge passed → Funded) are detected automatically by comparing the current balance watermark against phase-definition thresholds stored in the `phase_definitions` table.

CHAPTER 7: DEPLOYMENT & OPERATIONS

7.1 Production Stack

Component	Technology	Detail
OS	Ubuntu 22.04 LTS	VPS or dedicated server
WSGI Server	Gunicorn 21+	4 workers, gevent worker class
Proxy	Nginx	SSL termination, static files, rate limiting
SSL	Let's Encrypt / Certbot	Auto-renewal via systemd timer
DB (prod)	PostgreSQL 15	Managed instance; SQLite used for dev
Process Mgmt	systemd	Auto-restart on crash, boot-time start
Secrets	Environment vars	FLASK_SECRET_KEY, DATABASE_URL, etc.

7.2 Gunicorn Configuration

■ gunicorn.conf.py

```
workers = 4 worker_class = 'gevent' bind = '127.0.0.1:5000' timeout = 120 keepalive = 5
max_requests = 1000 max_requests_jitter = 50 accesslog = 'logs/access.log' errorlog =
'logs/error.log' loglevel = 'info'
```

7.3 Nginx Reverse Proxy (sample)

■ nginx site configuration (condensed)

```
server { listen 443 ssl; server_name yourdomain.com; ssl_certificate
/etc/letsencrypt/live/yourdomain.com/fullchain.pem; ssl_certificate_key
/etc/letsencrypt/live/yourdomain.com/privkey.pem; location / { proxy_pass
http://127.0.0.1:5000; proxy_set_header Host $host; proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for; proxy_set_header X-Forwarded-Proto
$scheme; proxy_read_timeout 120s; } location /static/ { alias /var/www/dashboard/static/;
expires 7d; add_header Cache-Control "public, immutable"; } } # HTTP -> HTTPS redirect server {
listen 80; server_name yourdomain.com; return 301 https://$host$request_uri; }
```

7.4 Environment Variables

Variable	Required	Description
FLASK_SECRET_KEY	Yes	Random 64-char hex string for session signing
DATABASE_URL	Prod	PostgreSQL connection string (postgres://...)
FLASK_ENV	No	'production' disables debug mode
RATE_LIMIT_STORAGE	No	Redis URI for distributed rate limiting
LOG_LEVEL	No	'DEBUG' 'INFO' 'WARNING'

7.5 Deployment Script

■ deploy.sh – production deploy (condensed)

```
#!/bin/bash set -e echo "[1/5] Pulling latest code..." git pull origin main echo "[2/5] Installing Python dependencies..." pip install -r requirements-production.txt echo "[3/5] Running database migrations..." python migrations.py echo "[4/5] Collecting static files..." # (No build step needed – static assets served directly by Nginx) echo "[5/5] Reloading Gunicorn..." systemctl reload gunicorn echo "Deploy complete."
```

7.6 Rate Limiting Strategy

Flask-Limiter applies layered rate limits to prevent abuse. API push endpoints allow higher throughput for the Trader Companion app, while authentication endpoints are tightly restricted to block brute-force attacks.

Endpoint Group	Limit	Rationale
/api/auth/login	5/minute	Prevent credential stuffing
/api/update_data	60/minute	Allow frequent MT5 sync pushes
/api/get_data	120/minute	Dashboard polling
Default (all others)	10 000/day, 2 000/hour	General protection

CHAPTER 8: SECURITY ARCHITECTURE

8.1 Defence in Depth

Security is implemented at multiple layers: transport (TLS), network (Nginx rate limiting + firewall), application (RBAC decorators + brute-force lockout), and data (PBKDF2 password hashing + parameterised SQL queries).

8.2 Brute-Force Protection

The `login_attempts` table records every failed login. After 5 consecutive failures from the same IP or for the same username, the account is locked for 15 minutes. The lockout is checked via `is_account_locked()` before any credential verification occurs.

■ `dashboard/database.py` – `login_attempts` table

```
CREATE TABLE IF NOT EXISTS login_attempts ( id INTEGER PRIMARY KEY AUTOINCREMENT, identifier TEXT NOT NULL, -- username or IP attempt_time TEXT NOT NULL, success INTEGER DEFAULT 0 );
```

8.3 SQL Injection Prevention

All database queries use parameterised statements (`cursor.execute(sql, (param,))`). No string formatting or concatenation is used for SQL construction anywhere in the codebase.

■ `dashboard/database.py` – parameterised query (example)

```
# CORRECT – parameterised cursor.execute( 'SELECT * FROM user_credentials WHERE username = ? AND user_type = ?', (username, user_type) ) # NEVER done – vulnerable to injection # cursor.execute(f"SELECT * FROM user_credentials WHERE username = '{username}'")
```

8.4 Session Management

Sessions are stored in the database as a `sessions` table. Each session token is a 64-character hex string generated by `secrets.token_hex(32)`. Sessions expire after 24 hours of inactivity. Tokens are transmitted exclusively via `HttpOnly` cookies to prevent JavaScript access.

8.5 Audit Log

Every significant action — login, data update, rollback, access denial, API key generation — is written to the `audit_log` table with the timestamp, action type, user identity, IP address, and success flag. Super admins can query the full audit log via `/api/audit_log`.

CHAPTER 9: DEVELOPER WORKFLOWS

9.1 Local Development Setup

■ Setup commands (Windows PowerShell)

```
# Clone repository git clone https://github.com/Oukaharry/FuturesAutomatedFeed.git cd
FuturesAutomatedFeed # Create virtual environment python -m venv .venv
.venv\Scripts\Activate.ps1 # Install dependencies pip install -r requirements.txt # Initialise
database python -c "from dashboard.database import init_database; init_database()" # Run
development server python wsgi.py # → http://localhost:5000
```

9.2 Common Debug Scripts

Script	Purpose
debug_fetch.py	Test /api/get_data against a running server
debug_ev.py	Inspect evaluation matching for a specific account
debug_phase_logic.py	Trace phase-transition detection logic
debug_compare.py	Diff two client data JSON snapshots
debug_waterlog.py	Inspect watermark records for a client
check_audit.py	Print recent audit log entries
check_db_table.py	Dump any SQLite table to stdout
reproduce_match.py	Reproduce an account-matching result with test data
quick_test.py	Run lightweight sanity checks against the server

9.3 Git Workflow

■ Standard commit workflow

```
git add -A git commit -m "feat: descriptive message" git push origin main # Check remote status
git log --oneline -5 git status
```

9.4 Building the Desktop App

■ PyInstaller build (from workspace root)

```
# Ensure PyInstaller is installed pip install pyinstaller # Build using the spec file
pyinstaller Trader_Companion_v1.2.0.spec --clean --noconfirm # Output location #
dist/Trader_Companion_v1.2.0/Trader_Companion_v1.2.0.exe
```

CHAPTER 10: REST API REFERENCE

10.1 Authentication

The API supports two authentication mechanisms: (1) **Session Cookie** — set on login via `/api/auth/login` and sent automatically by the browser for all subsequent requests; (2) **API Key** — a 32-char hex token sent in the `X-API-Key` header, intended for the Trader Companion desktop app and programmatic access.

10.2 POST `/api/auth/login`

■ Request

```
POST /api/auth/login Content-Type: application/json { "username": "trader_harry", "password": "s3cur3P@ss", "user_type": "trader" }
```

■ Response (200 OK)

```
HTTP/1.1 200 OK Set-Cookie: session_token=abc123...; HttpOnly; SameSite=Strict { "status": "success", "user_type": "trader", "username": "trader_harry", "must_change_password": false }
```

10.3 GET `/api/get_data`

■ Request

```
GET /api/get_data?client_id=harry_client_1 X-API-Key: <your-api-key> # OR session cookie
```

■ Response schema (200 OK)

```
{ "status": "success", "deals": [ { "ticket": 123, "symbol": "NQ", "profit": 150.0, ... } ], "positions": [ { "ticket": 456, "symbol": "ES", "volume": 1.0, ... } ], "account": { "balance": 125000, "equity": 124800, "login": 9876543 }, "evaluations": [ { "Firm": "Topstep", "Account #": "TSX123456", ... } ], "statistics": { "hedging_review": { ... }, "account_stats": { ... } }, "last_updated": "2025-01-15T14:32:00" }
```

10.4 POST `/api/update_data`

■ Request

```
POST /api/update_data Content-Type: application/json X-API-Key: <your-api-key> { "identity": { "client": "harry_client_1", "trader": "trader_harry" }, "deals": [ ... ], "positions": [ ... ], "account": { ... }, "evaluations": [], # empty = keep server-side evaluations "statistics": { ... } }
```

■ Response (200 OK)

```
{ "status": "success", "message": "Data updated", "version": 42 # new version number in history }
```

10.5 POST `/api/rollback`

■ Request

```
POST /api/rollback Content-Type: application/json Cookie: session_token=... { "client_id": "harry_client_1", "version": 38 # target version to restore }
```

■ Response (200 OK)

```
{ "status": "success", "message": "Rolled back to version 38", "new_version": 43 # restoration itself creates a new version }
```


10.6 Error Responses

HTTP Status	When / Meaning
400 Bad Request	Missing required fields in request body
401 Unauthorized	No session cookie and no API key provided
403 Forbidden	Valid credentials but insufficient permissions for resource
404 Not Found	Requested client_id exists but has no saved data yet
429 Too Many Requests	Rate limit exceeded (see per-endpoint limits)
500 Internal Server Error	Unhandled exception — check server log

CHAPTER 11: COMPLETE FILE INVENTORY

11.1 Backend — Core Modules

File	Lines	Purpose
dashboard/app.py	~4 434	Flask application — all routes, auth decorators, business logic
dashboard/database.py	~1 303	All DB operations — CRUD, sessions, API keys, history, rollback
dashboard/financial_overview.py	~1 400	Analytics engine — P&L, payout, deposit, fee, hedge calculations
dashboard/scheduler.py	~80	Background thread that records midnight watermarks daily
dashboard/notes_service.py	~70	Inline cell note CRUD backed by SQLite notes table
dashboard/watermark_service.py	~280	Watermark + waterlog period persistence and aggregation
dashboard/phase_manager.py	~250	Phase lifecycle management — create, complete, chain phases
dashboard/email_service.py	~—	Email notification helper (SMTP)

11.2 Backend — Utility Modules

File	Purpose
dashboard/utills/trade_matcher.py	UnifiedTradeMatcher class — segment deals into sessions and match to eval rows
dashboard/utills/sheet_helper.py	Helper functions for spreadsheet column mapping and formula replication
dashboard/api_client.py	Internal API client helper used by some admin scripts
dashboard/calc_like_sheet.py	Replicates spreadsheet financial formulas in Python for verification
dashboard/financial_overview.py	SimpleCache, cache_result decorator, and all analytics functions

11.3 Config Modules

File	Purpose
config/hierarchy.py	Load/save/mutate the 4-tier admin→trader→client JSON hierarchy; also holds SYSTEM_HIERARCHY global
config/hierarchy.json	Persistent JSON file storing the entire user/client organisational tree
config/settings.py	Application-level settings (DB path, secret key, rate limits)
config/production.py	Production overrides (PostgreSQL URL, Gunicorn workers, etc.)

11.4 Desktop Application

File	Purpose
trader_companion/trader_app.py	~2 400 lines — MT5DataPusher class + TraderCompanionApp Tkinter UI

11.5 Frontend Templates & Static Assets

File	Purpose
dashboard/templates/index.html	~4 300 lines — main single-page dashboard (evaluations, stats, hedging, history)
dashboard/templates/login.html	Login page for all user types
dashboard/templates/super_admin.html	Super-admin management panel
dashboard/templates/admin_dashboard.html	Admin view — trader + client management
dashboard/templates/financial_overview.html	Financial overview charts (Chart.js)
dashboard/static/css/style.css	Full dark-theme design system (~1 000 lines)

11.6 Build & Deployment Scripts

File	Purpose
wsgi.py	Gunicorn WSGI entry point
gunicorn.conf.py	Gunicorn worker, timeout, and logging config
deploy.sh	Linux production deployment shell script
deploy.ps1	Windows deployment PowerShell script
build.py	Helper script to trigger PyInstaller builds
migrations.py	Database schema migration runner
prepare_deployment.py	Copies files into deployment_package/ folder
requirements.txt	Development Python dependencies
requirements-production.txt	Production Python dependencies (no debug packages)
Trader_Companion_v1.2.0.spec	PyInstaller spec for the desktop app

11.7 Debugging & Utility Scripts

Script	Purpose
debug_fetch.py	Test /api/get_data against a running server and print result
debug_fetch_test.py	Extended fetch debugging with header/cookie inspection
debug_ev.py	Inspect evaluation matching for a specific account number
debug_phase_logic.py	Trace phase-transition detection with sample data
debug_compare.py	Diff two client data JSON snapshots field-by-field
debug_waterlog.py	Inspect watermark records for a client from DB

Script	Purpose
debug_json.py	Pretty-print raw JSON from the dashboard DB
debug_db_match.py	Check DB for account-match candidates
debug_parser.py	Run deal comment parser against sample comments
debug_show_eval.py	Print evaluation rows with computed account signatures
debug_find_account.py	Search all clients for a given MT5 account number
debug_publish.py	Force-push a data payload to the dashboard API
check_audit.py	Print recent audit log entries from DB
check_db_details.py	Show full DB record for a specific client
check_db_table.py	Dump any SQLite table to stdout
check_watermarks.py	Print watermark records for all clients
check_fees.py	Verify fee calculations against spreadsheet formulas
reproduce_match.py	Reproduce an account-matching result with test data
reproduction_script.py	Full reproduction of data pipeline with sample input
quick_test.py	Run lightweight sanity checks against the server
manage_users.py	CLI script to create/list/delete users in the DB
migrate_data.py	One-time migration of data between DB schemas
trigger_migration.py	Trigger pending migrations programmatically
update_hierarchy_data.py	Sync hierarchy.json with DB user_credentials table

CHAPTER 12: APP.PY — COMPLETE FUNCTION REFERENCE

12.1 Helper & Utility Functions

Function	Line	Description
get_account_signature(account_number)	100	Extract first-4+last-4 digits signature; handles PREFIX...SUFFIX truncated format
get_last_n_digits(account, n=5)	133	Extract last N digits from an account number string using regex
match_account_to_evaluation(account_number, evaluations, phase_code)	156	Return list of (eval_index, matched_account) for all matching eval rows
parse_sheet_date(date_str)	271	Parse date strings in M/D/YYYY, YYYY-MM-DD, and ISO8601 formats
filter_matches_by_date(matches, evaluations, trade_timestamp, phase_code, trade_number)	320	Filter candidate eval matches by comparing trade date against eval start date
normalize_account_size(value)	481	Normalise account size strings: strip '\$','K','k' and convert to float
normalize_evaluations(evaluations)	519	Apply normalize_account_size to the 'Account Size' field of every eval row
get_field_name_for_phase(phase_code, trade_number, farming_date, evaluations, eval_idx, account_number)	531	Determine which column (Prop Day N / Trade N / Funded Profit) to update for a given trade
update_evaluations_from_aggregated_data(evaluations, aggregated_data, raw_deals)	585	Core matching engine: walk aggregated deal groups and write profit values into eval rows
init_admin_password()	1652	Seed the super_admin password from environment on first boot
get_filtered_hierarchy(user_type, user_identifier)	1937	Return hierarchy subtree visible to the calling user based on their role
can_manage_user(manager_type, manager_identifier, target_username, target_user_type)	2894	Check if a manager is allowed to reset/deactivate a target user
can_access_client(user_type, user_identifier, target_client)	3473	Return True if the caller is allowed to read/write data for target_client
get_accessible_clients(user_type, user_identifier)	3497	Return list of all client IDs the caller can access
update_data_with_api_key(data, identity, user_info)	3935	Handle /api/update_data when called with API-key instead of session cookie
run_dashboard()	4419	Start Flask development server (called from wsgi.py)

12.2 Auth Decorators

Decorator	Line	Description
require_api_key(f)	1664	Validate X-API-Key header; inject request.api_user; log access
require_admin_password(f)	1686	Validate admin_password in body or X-Admin-Password header
require_role(*allowed_roles)	1711	Session-cookie gate; check user_type against whitelist; return 403 if denied
require_session(f)	1736	Session-cookie gate for HTML pages; redirect to / if not logged in

12.3 Web (HTML) Routes

Route / Function	Line	Auth	Returns
GET / → index()	1756	None	login.html or redirect to dashboard
GET /super_admin → super_admin()	1777	Session	super_admin.html
GET /admin/ → admin_dashboard()	1784	Session	admin_dashboard.html
GET /financial_overview → financial_overview()	1796	Session (super_admin)	financial_overview.html with chart data
GET /payout_history → payout_history()	1838	Session	payout_history.html
GET /client_performance → client_performance()	1880	Session	client_performance.html
GET /trader_performance → trader_performance()	1888	Session	trader_performance.html
GET /trader/ → trader_dashboard()	1900	Session	index.html (trader view)
GET /dashboard/ → client_dashboard()	1915	Session	index.html (client view)
GET /super_admin/clients → client_management()	3466	Session	client management template
GET /change-password → change_password_page()	3033	Session	change_password.html
GET /logout → logout()	2675	None	Deletes session cookie, redirects to /

12.4 Authentication API Routes

Route / Function	Line	Description
POST /api/login → api_login()	2641	Username+password login for admin/trader; sets session cookie
POST /api/admin_login → api_admin_login()	2658	Legacy admin-only login endpoint
POST /api/auth/login → unified_login()	2715	Unified login for all roles; detects user type automatically
POST /api/auth/check-admin → check_admin_identifier()	2701	Check if a username exists as an admin (pre-login hint)
POST /api/logout → api_logout()	2687	Invalidate session token in DB

Route / Function	Line	Description
POST /api/auth/change_password → api_change_password()	3040	Change own password (requires current password + new password)
POST /api/admin/change_password → change_admin_password()	4339	Super-admin reset password for any user (legacy)
POST /api/user/reset_password → api_reset_password_rbac()	2941	RBAC-aware password reset (admin can reset traders; trader can reset clients)
POST /api/admin/reset_password → api_reset_password()	2982	Force-reset a user's password to a random value

12.5 User Management API Routes

Route / Function	Line	Description
POST /api/admin/create_user → api_create_user()	2805	Create a new user (any type) with initial password
GET /api/admin/list_users → api_list_users()	2933	Return all users visible to the caller's role
POST /api/admin/deactivate_user → api_deactivate_user()	3014	Soft-deactivate a user account (is_active=0)
POST /api/add_admin → api_add_admin()	3085	Add admin to hierarchy.json and DB
POST /api/add_trader → api_add_trader()	3173	Add trader under a given admin
POST /api/add_client → api_add_client()	3194	Add client under a given trader
POST /api/update_admin → api_update_admin()	3129	Update admin name/email in hierarchy
POST /api/update_trader → api_update_trader()	3141	Update trader name/email in hierarchy
POST /api/update_client → api_update_client()	3154	Update client name/email/category in hierarchy
POST /api/update_client_profile → api_update_client_profile()	3295	Update client dashboard-visible profile fields
POST /api/remove_admin → api_remove_admin()	3333	Remove admin and all subordinates from hierarchy
POST /api/remove_trader → api_remove_trader()	3343	Remove trader and all subordinate clients
POST /api/remove_client → api_remove_client()	3354	Remove client from hierarchy
POST /api/delete_user → api_delete_user()	3097	Hard-delete user credential from DB
POST /api/move_client → api_move_client()	3449	Move client to a different trader/admin
POST /api/move_trader → api_move_trader()	3457	Move trader to a different admin
POST /api/move_user → api_move_user()	3234	Generic user move with hierarchy validation

12.6 Data API Routes

Route / Function	Line	Description
GET /api/data → get_data()	3700	Fetch client JSON data; supports session + API-key auth; injects notes

Route / Function	Line	Description
POST /api/update_data → update_data()	3842	Push new data snapshot; merge with existing; save versioned history
POST /api/notes → update_note()	3783	Save or blank-delete an inline cell note for a row/column
POST /api/notes/delete → delete_note()	3826	Hard-delete an inline cell note
GET /api/hierarchy → get_hierarchy()	2004	Return filtered hierarchy tree for the caller
GET /api/super_admin/totals → get_super_admin_totals()	2035	Aggregate totals (accounts, balances, P&L;) across all clients
POST /api/client/update_source → update_client_source()	2126	Update the data source tag for a client
POST /api/client/lookup → api_client_lookup()	2181	Look up a client by email for the desktop app login flow
POST /api/client/auth → api_client_auth()	2204	Authenticate a client from the desktop app
POST /api/client/push → api_client_push()	2260	Large push endpoint used by Trader Companion (evaluations + deals + positions)
POST /api/client/migrate_sheet → api_migrate_sheet()	2443	Import a Google Sheet CSV export into the client's evaluation table
GET /api/client/watermark_history/ → api_get_watermark_history()	2570	Return daily watermark records for an eval index
POST /api/hedging_review/ → update_hedging_review()	3533	Save/update the hedging review section (deposits, withdrawals, hedge result)
POST /api/historical_mt5/ → manage_historical_mt5()	3592	Add, update, or delete historical MT5 account records
POST /api/client/delete_evaluation → api_delete_evaluation()	3381	Soft-delete (or hard-delete) individual evaluation row

12.7 Version History & Rollback Routes

Route / Function	Line	Description
POST /api/client/history → api_get_client_history()	4134	List all version snapshots for a client (id, version, action, changed_by, timestamp)
POST /api/client/version → api_get_client_version()	4164	Fetch the full JSON snapshot for a specific version number
POST /api/client/rollback → api_rollback_client_data()	4207	Restore client data to a past version; creates new version N+1
POST /api/client/compare_versions → api_compare_versions()	4261	Return diff summary between two versions
GET /api/admin/all_history → api_get_all_history()	4297	Return history records across ALL clients (admin+ only)

12.8 API Key Management Routes

Route / Function	Line	Description
POST /api/admin/generate_key → api_generate_key()	4018	Generate a new API key scoped to a trader+client pair
GET /api/admin/list_keys → api_list_keys()	4043	List all active API keys (prefixes only, not full keys)
POST /api/admin/revoke_key → api_revoke_key()	4051	Revoke an API key by its prefix
GET /api/admin/audit_log → api_audit_log()	4068	Return last N audit log entries, filterable by action type

12.9 Trader Push Routes (Granular)

Route / Function	Line	Description
POST /api/trader/push_account → push_account_data()	4081	Push just the account summary object for a client
POST /api/trader/push_positions → push_positions()	4094	Push just the open positions array
POST /api/trader/push_deals → push_deals()	4107	Push just the closed deals history array
POST /api/trader/push_evaluations → push_evaluations()	4120	Push just the evaluations array (replaces existing)

12.10 System / Stats Routes

Route / Function	Line	Description
GET /health /api/health → health_check()	4326	Return {'status': 'healthy'} plus DB and key counts — used by load balancer probes
GET /api/sheet/stats → get_stats_sheet_data()	4364	Return account statistics formatted for the Stats sheet tab
GET /api/sheet/waterlog → get_waterlog_sheet_data()	4376	Return waterlog period data for all clients

CHAPTER 13: DATABASE.PY — COMPLETE FUNCTION REFERENCE

13.1 Connection & Initialisation

Function	Line	Description
get_db_path() → str	16	Return DB file path from DATABASE_URL env var or default dashboard.db
get_connection() →	20	Context manager returning a sqlite3/psycopg2 connection with Row factory
init_database()	29	Create all 15+ tables if not exist; run column migrations; seed super_admin

13.2 Password & Admin

Function	Line	Description
hash_password(password, salt=None) → (hash, salt)	247	PBKDF2-SHA256, 100 000 iterations; generate salt if not provided
verify_password(password, stored_hash, salt) → bool	262	Recompute hash and compare with secrets.compare_digest (timing-safe)
set_admin_password(username, password) → bool	269	Upsert hashed password for a super_admin or admin user
verify_admin_password(username, password) → bool	291	Verify password for admin accounts specifically

13.3 User CRUD

Function	Line	Description
create_user(username, password, user_type, email, parent_admin, parent_trader) → bool	308	Insert new user_credentials row; idempotent (returns False if exists)
verify_user_password(username, user_type, password) → dict None	332	Verify credentials; stamp last_login; return user dict or None
verify_client_login(email, password) → dict None	367	Verify client by email (clients log in with email, not username)
update_user_password(username, user_type, new_password) → bool	402	Hash and store a new password; set must_change_password=0
get_user(username, user_type) → dict None	417	Fetch a user record without password verification
list_users(user_type=None) → list	430	List all (or type-filtered) users; excludes password fields
deactivate_user(username, user_type) → bool	449	Set is_active=0 (soft delete — user cannot log in but record preserved)

Function	Line	Description
activate_user(username, user_type) → bool	460	Re-enable a deactivated user
reset_user_password(username, user_type) → str	471	Generate random 12-char password, hash and store it, return plaintext
find_user_by_identifier(identifier) → dict None	490	Search by username or email across all user types
verify_user_by_identifier(identifier, password) → dict None	512	Flexible login using email or username
delete_user_credential(username, user_type) → bool	551	Hard-delete a user credential row
update_user_email(username, user_type, new_email) → bool	562	Update the email address for a user
user_exists(username, user_type) → bool	574	Quick existence check without fetching the full record

13.4 Brute-Force Protection

Function	Line	Description
record_login_attempt(username, user_type, ip_address, success)	584	Insert a row into login_attempts with current timestamp
get_failed_login_count(username, user_type, minutes=15) → int	594	Count failed attempts within the last N minutes
is_account_locked(username, user_type, max_attempts=5) → bool	608	Return True if failed count >= max_attempts in last 15 min

13.5 API Key Management

Function	Line	Description
hash_api_key(api_key) → str	614	SHA-256 hash a key for storage (raw key never stored)
generate_api_key(admin, trader, client="") → str	618	Generate 64-char hex key; store hash + metadata; return plaintext
validate_api_key(api_key) → dict None	638	Hash incoming key and look up in DB; return metadata or None
list_api_keys() → list	667	Return all API key records (prefix + metadata, no raw keys)
revoke_api_key(key_prefix) → bool	677	Set is_active=0 for key matching prefix
delete_api_key(key_prefix) → bool	688	Hard-delete API key record

13.6 Client Data Storage

Function	Line	Description
save_client_data(client_id, data, overwrite=False) → bool	698	Upsert full client JSON blob to clients_data; each field stored in separate column

Function	Line	Description
get_client_data(client_id) → dict None	798	Fetch and deserialise all JSON columns for a client
get_all_clients() → dict	827	Return {client_id: data} mapping for every client in DB
get_clients_count() → int	838	Return total number of clients in DB
update_client_field(client_id, field, value) → bool	846	Update a single field (column) in clients_data without touching others

13.7 Version History

Function	Line	Description
get_next_version(client_id) → int	874	Return max(version)+1 for a client from data_history
save_data_snapshot(client_id, data, action, changed_by, ...) → int	890	Write a new version row to data_history; return version number
save_client_data_with_history(client_id, data, action, ...) → (bool, int)	944	save_client_data + save_data_snapshot in one atomic call
get_data_history(client_id, limit=50) → list	969	Return list of version metadata dicts (newest first)
get_data_version(client_id, version) → dict None	988	Fetch and deserialise a specific snapshot from data_history
rollback_to_version(client_id, version, changed_by, ...) → (bool, int)	1022	Fetch target snapshot and save it as new version N+1
compare_versions(client_id, version1, version2) → dict	1051	Return field-level diff between two snapshots
get_latest_version(client_id) → int	1103	Return the current highest version number for a client
cleanup_old_history(client_id=None, keep_versions=100) → int	1113	Delete old snapshots beyond keep_versions threshold; return count deleted

13.8 Audit Log

Function	Line	Description
log_action(action, user_type, user_identifier, ip_address, description="", success=True)	1160	Insert a row into audit_log with current timestamp
get_audit_log(limit=100, action_filter=None) → list	1179	Return recent audit log rows, optionally filtered by action type

13.9 Session Management

Function	Line	Description
create_session(user_type, user_identifier, ip_address, user_agent) → str	1200	Insert session row; return 64-char hex token as HttpOnly cookie value

Function	Line	Description
validate_session(session_token) → dict None	1217	Look up session token; check expiry; return user info or None
delete_session(session_token)	1241	Remove session row (logout)
cleanup_expired_sessions()	1248	Delete all sessions older than 24 hours (run periodically)

13.10 Migration Utility

Function	Line	Description
migrate_from_json(api_keys_file, data_file)	1260	One-time import of legacy JSON-file data into SQLite tables

CHAPTER 14: FINANCIAL_OVERVIEW.PY — FUNCTION REFERENCE

14.1 Infrastructure / Cache

Function / Class	Line	Description
class SimpleCache	10	Thread-safe TTL cache backed by a dict; used to memoize expensive aggregations
cache_result(ttl=300)	46	Decorator factory — wraps a function with cache lookup and 5-min TTL
_get_cached_clients()	65	Return all client data from DB, memoized for 60 s to avoid repeated DB reads
clear_financial_cache()	100	Invalidate all entries in the SimpleCache instance
col_idx_to_letter(n)	34	Convert 0-based column index to Excel-style letter(s): 0→A, 26→AA, etc.

14.2 Data Normalisation Helpers

Function	Line	Description
parse_currency(value_str)	73	Strip '\$', ',', 'K' and convert to float; return 0.0 on failure
normalize_prop_firm_name(name)	418	Map raw firm name strings to canonical names: 'ftmo', 'topstep', 'lucid', etc.
parse_date(date_str)	469	Parse date strings from multiple format patterns; return datetime or None
_aggregate_events_cumulative(events)	1201	Sort events by date and compute running cumulative sum

14.3 Payout & Growth Functions

Function	Line	Description
get_payouts_history(start_date, end_date, prop_firm_filter, profile_filter) → list	483	Scan all client evaluations for payout entries; return sorted event list
get_payouts_growth_data(profile_filter) → list	582	Return time-series of cumulative payout value
get_portfolio_growth_data(profile_filter) → list	847	Combine payout + trading profit + deposit into running portfolio value

14.4 MT5 Deal Data Functions

Function	Line	Description
get_mt5_deals_data(profile_filter) → list	642	Return all closed deals across all clients with parsed profit/commission/swap

Function	Line	Description
get_cumulative_deposits(profile_filter) → list	708	Time-series of running total deposits from MT5 deal history
get_cumulative_trading_profit(profile_filter) → list	741	Time-series of running realised P&L; from closed trades

14.5 Overhead & Hedge Functions

Function	Line	Description
get_cumulative_fees_data(profile_filter) → list	1094	Time-series of running evaluation fees paid across all prop firms
get_cumulative_hedge_data(profile_filter) → list	1131	Time-series of running hedge account P&L; vs sheet-stated hedge result
get_cumulative_farming_data(profile_filter) → list	1171	Time-series of running farming phase profit

14.6 Aggregate Overview Functions

Function	Line	Description
calculate_propfirm_overview(profile_filter) → dict	933	Aggregate per-firm: active accounts, total deposits, total payouts, net P&L; ROI
calculate_trader_stats(profile_filter) → list	1226	Per-trader breakdown: win rate, average trade, max DD, Sharpe-like ratio
get_client_performance_stats(profile_filter) → list	1361	Per-client breakdown: funded phases, payouts, current balance
calculate_all_financials(profile_filter) → dict	105	Master wrapper: calls all calculation functions; returns combined dict

14.7 Data Access Helper

Function	Line	Description
get_cached_clients_dataset()	65	Return memoized dataset of all client data from DB (shortcut used by most analytics functions)

CHAPTER 15: TRADER COMPANION — FULL FUNCTION REFERENCE

15.1 MT5DataPusher Class

MT5DataPusher (line 58) is the data-layer class responsible for all MT5 interactions and server communication. It is instantiated by `TraderCompanionApp` and lives for the lifetime of the application session.

Method	Line	Description
<code>__init__(self, dashboard_url, api_key)</code>	61	Store server URL and API key; initialise MT5 connection state
<code>connect_mt5(self, login, password, server, terminal_path) → (bool, str)</code>	68	Call <code>mt5.initialize()</code> + <code>mt5.login()</code> ; return (success, message)
<code>disconnect_mt5(self)</code>	105	Call <code>mt5.shutdown()</code> and clear connection state
<code>get_account_info(self) → dict</code>	112	Fetch account summary via <code>mt5.account_info()</code> ; serialise <code>NamedTuple</code> to dict
<code>get_positions(self) → list</code>	156	Fetch open positions via <code>mt5.positions_get()</code> ; convert each to dict
<code>get_deals(self, days=30) → list</code>	184	Fetch deal history from (now-days) to now; convert to JSON-serialisable list
<code>_deal_type_to_string(self, deal_type) → str</code>	235	Convert MT5 deal type enum int to string: 'BUY', 'SELL', 'BALANCE', etc.
<code>_entry_to_string(self, entry) → str</code>	240	Convert MT5 deal entry enum to 'IN', 'OUT', 'INOUT', 'OUT_BY'
<code>calculate_statistics(self, deals) → dict</code>	244	Compute <code>win_rate</code> , <code>avg_profit</code> , <code>total_profit</code> , <code>drawdown</code> from deal list
<code>parse_deal_comment_v2(self, comment) → dict</code>	272	Parse MT5 comment string into { <code>phase_code</code> , <code>account_num</code> , <code>trade_num</code> , <code>date_str</code> }
<code>aggregate_deals_by_comment_v2(self, deals) → list</code>	296	Group deals by parsed comment key; sum profits per group
<code>get_deals_grouped_by_phase(self, days=365) → dict</code>	310	Partition deals into { <code>'challenge':[]</code> , <code>'funded':[]</code> , <code>'hedge':[]</code> , <code>'farming':[]</code> }
<code>parse_deal_comment(self, comment) → dict</code>	361	Legacy comment parser — original regex-based implementation
<code>aggregate_deals_by_account(self, deals) → dict</code>	423	Group deals by account number key; compute totals per account
<code>push_to_dashboard(self, client_name, admin_name, trader_name) → dict</code>	475	Build full payload (account+deals+positions+evaluations+stats) and POST to server
<code>extract_account_core(self, account_num) → str</code>	598	Extract core identifier from account number for matching

Method	Line	Description
<code>process_deals_for_evaluations(self, deals, evaluations) → list</code>	623	Orchestrator: choose new or legacy parser based on comment format
<code>_process_deals_with_new_parser(self, deals, evaluations) → list</code>	653	New V2 parser pipeline: parse → segment → match → write to eval rows
<code>_find_evaluation_match(self, account_number, phase_code, eval_lookup) → int None</code>	804	Lookup eval row index from pre-built account/phase map
<code>_get_field_name_for_phase(self, phase_code, trade_number, farming_date, evaluations, eval_idx, forced_day_num) → str None</code>	827	Return column key (e.g. 'Prop Day 3', 'Trade 2') for a given phase+trade combo
<code>_calculate_farming_day(self, farming_date_str, evaluations, eval_idx) → int None</code>	878	Compute which Prop Day N column the farming trade belongs to
<code>_process_deals_legacy(self, deals, evaluations) → list</code>	917	Legacy matching pipeline using the original phase-code approach

15.2 TraderCompanionApp Class

TraderCompanionApp (line 1024) is the Tkinter GUI class. It owns the window, all widgets, the auto-push timer, and delegates data operations to an **MT5DataPusher** instance.

Method	Line	Description
<code>__init__(self)</code>	1027	Create root Tk window; init MT5DataPusher; load config; call <code>setup_ui()</code>
<code>setup_ui(self)</code>	1076	Build all Tkinter widgets: gradient header, server selector, MT5 login form, log area, buttons
<code>log(self, message, level='INFO')</code>	1298	Append timestamped message to the scrolled log <code>TextWidget</code>
<code>lookup_client(self)</code>	1306	POST to <code>/api/client/lookup</code> with entered email; populate client fields on success
<code>toggle_mt5_connection(self)</code>	1371	Connect or disconnect MT5 based on current state; update button label
<code>push_data(self)</code>	1387	Full data push: <code>get_account_info</code> → <code>get_deals</code> → <code>process_deals_for_evaluations</code> → <code>push_to_dashboard</code>
<code>push_mt5_only(self)</code>	1503	Push only account+deals+positions without evaluation processing
<code>show_deal_comments(self)</code>	1620	Open popup window listing all deal comments found in MT5 history
<code>sync_hedge_results(self)</code>	1674	Fetch current evaluations from server → find hedge deals → update hedge fields → push back
<code>analyze_comments_v2(self)</code>	1800	Run V2 comment parser on all deals and display analysis in popup
<code>show_aggregated_data(self)</code>	1906	Show popup table of deals aggregated by comment/account
<code>push_by_comment(self)</code>	1971	Advanced push: group deals by comment, match to eval rows, push result

Method	Line	Description
migrate_from_sheet(self)	2125	POST sheet CSV to /api/client/migrate_sheet to import evaluation data
verify_stats(self, local_stats, dashboard_stats) → list	2210	Compare locally computed stats with server-stored stats; return list of differences
toggle_auto_push(self)	2258	Enable/disable the auto-push background loop thread
check_and_push_update(self)	2283	Compare current MT5 data hash with last-pushed hash; push only if changed
auto_push_loop(self)	2326	Background thread function: sleep → check_and_push_update → repeat
save_config(self)	2338	Persist server URL, client, trader, admin, API key to trader_config.json
load_config(self)	2355	Load saved config from trader_config.json and populate UI fields
run(self)	2389	Enter Tkinter mainloop; called by main()

15.3 Module-Level Functions

Function	Line	Description
main()	2394	Entry point: create TraderCompanionApp and call run()

CHAPTER 16: SUPPORTING MODULES — FUNCTION REFERENCE

16.1 dashboard/scheduler.py

Function	Line	Description
run_scheduler()	10	Infinite loop: sleep until next midnight UTC, then call update_all_clients_watermarks()
update_all_clients_watermarks()	34	Iterate all clients; for each active evaluation compute today's balance watermark and write to DB
start_scheduler()	64	Spawn run_scheduler() in a daemon background thread; called once on Flask app startup

16.2 dashboard/notes_service.py

Function	Line	Description
get_client_notes(client_id) → dict	4	Return {row_index: {col_key: text}} mapping of all notes for a client
save_client_note(client_id, row_index, column_key, content, user)	39	Upsert a note for a specific row+column; record who last edited it
delete_client_note(client_id, row_index, column_key) → bool	56	Delete a specific note cell from the notes table

16.3 dashboard/watermark_service.py

Function	Line	Description
save_daily_profit(client_id, net_profit, date_str, source) → bool	5	Upsert daily net profit for a client in daily_profits table
get_watermark_history(client_id, days=30) → list	37	Return last N days of balance watermark records for a client
get_lower_watermark(client_id, days=14) → float	58	Return the minimum balance seen in the last N days
get_high_watermark(client_id, days=14) → float	71	Return the maximum balance seen in the last N days
get_bulk_watermarks(days=14) → dict	84	Return {client_id: {high, low, current}} for all clients
get_aggregate_watermarks(days=14) → dict	125	Aggregate watermarks across all clients into a single summary
bulk_save_history(client_id, history_data)	166	Batch insert watermark records from a list of {date, balance} dicts
save_waterlog_periods(client_id, periods)	186	Persist array of waterlog period objects (start/end/peak/trough/max_dd)

Function	Line	Description
get_waterlog_periods(client_id) → list	210	Fetch all waterlog period records for a client
get_all_daily_watermarks(client_id) → list	228	Return every daily watermark row ever recorded for a client
compute_waterlog_from_db(client_id) → dict	254	Re-derive waterlog periods from raw daily watermark history in DB

16.4 dashboard/utils/trade_matcher.py — UnifiedTradeMatcher

Method	Line	Description
__init__(self, evaluations)	13	Store evaluations list; call _build_account_map()
_build_account_map(self)	24	Index evaluations by account number + phase for fast lookup during matching
_add_to_lookup(self, lookup, account_str, idx, date, acct_type)	51	Normalise and insert one account entry into the lookup dict
_parse_date(self, date_str) → datetime None	74	Parse date from eval row using multiple format attempts
process_deals(self, deals) → list	92	Main entry: parse comments → segment → match → apply; return updated evaluations
_segment_into_sessions(self, deals) → list	162	Group consecutive deals with same account+phase into trading sessions
_find_best_row_match(self, account_num, session_date) → int None	209	Return eval row index with best account+date overlap
_apply_session_to_row(self, row_idx, session_deals) → bool	277	Compute P&L; for the session and write into the correct column of the eval row
_get_field_name(self, phase, number) → str	311	Map (phase_code, trade_number) pair to column key string
_parse_deal_comment(self, deal) → dict	323	Regex-parse MT5 deal comment string into structured {phase, acct, num, date} dict
_parse_iso(self, date_str) → datetime None	356	Parse ISO 8601 timestamps with and without timezone

16.5 dashboard/phase_manager.py — PhaseManager

Method	Line	Description
initialize_default_phases()	13	@staticmethod — populate phase_definitions table with default Challenge/Funded/DoubleDip/Farming phases
create_evaluation(account_signature, firm, phase_code, start_date, ...)	70	Insert a new evaluations row with all metadata; return evaluation_id
complete_phase(evaluation_id, status, end_date) → int None	116	Mark phase as passed/failed; if passed, automatically create next-phase row and return its ID

Method	Line	Description
find_evaluation_for_trade(account_signature, trade_date_str) → dict None	194	Look up active evaluation row that matches account + trade date window
get_phase_chain(latest_evaluation_id) → list	226	Traverse prev_evaluation_id Links to return the full phase chain for an account

16.6 config/hierarchy.py

Function	Line	Description
load_hierarchy() → dict	7	Read hierarchy.json from disk; return parsed dict
reload_hierarchy()	15	Re-read hierarchy.json and update SYSTEM_HIERARCHY global in-place
save_hierarchy(hierarchy_data)	23	Write hierarchy dict back to hierarchy.json atomically
add_admin(admin_name, email="") → bool	—	Append admin entry to hierarchy; save
update_admin_details(admin_name, email)	—	Update admin email in hierarchy; save
update_trader_details(admin_name, trader_name, email)	—	Update trader email; save
update_client_details(admin, trader, client, email)	—	Update client email; save
update_client_category(admin, trader, client, category)	—	Set client category (e.g. 'vip', 'standard'); save
add_trader(admin_name, trader_name, email="") → bool	—	Append trader under admin; save
add_client(admin, trader, client, email="", category="")	—	Append client under trader; save
remove_admin(admin_name) → bool	—	Remove admin and all subordinates; save
remove_trader(admin_name, trader_name) → bool	—	Remove trader and clients; save
remove_client(admin, trader, client) → bool	—	Remove client entry; save
move_client(client, old_admin, old_trader, new_admin, new_trader) → bool	—	Move client to different trader/admin; save
move_trader(trader_name, old_admin, new_admin) → bool	—	Move trader with all clients to a different admin; save
get_client_profile(client_name) → dict None	—	Return single client metadata dict from hierarchy
get_client_by_email(email) → dict None	—	Search hierarchy by client email; return client profile
get_all_clients() → list	—	Flatten hierarchy to list of all client dicts
get_user_by_email(email) → dict None	—	Find any user (admin/trader/client) by email

CHAPTER 17: JAVASCRIPT FUNCTIONS — INDEX.HTML REFERENCE

All JavaScript in the dashboard lives in a single `<script>` block inside **dashboard/templates/index.html**. The ~2 800-line script block is organised into logical groups below.

17.1 Initialisation & Data Loading

Function	Line	Description
updateDashboard()	1949	Fetch <code>/api/data?client_id=...</code> , store in <code>currentData</code> , call all render functions
openTab(tabName)	1908	Switch visible tab (Evaluations / Stats / Positions / History / Hedging / Settings)
debugLog(msg)	1471	Write message to browser console when debug mode enabled

17.2 Evaluations Table

Function	Line	Description
renderEvaluationsTable(resetPage=true)	2063	Full table rebuild: filter deleted rows, sort desc by <code>originalIndex</code> , paginate, build HTML, insert into DOM
loadMoreEvaluations()	2318	Increment <code>evalPage</code> and call <code>renderEvaluationsTable(false)</code> to append next 50 rows
generateDayColumns(ev, cell, index, rowNum)	2376	Generate HTML cells for Prop Day 1...N columns including inline input and formula value
getColumnLetterForKey(key) → string	2326	Map column key name to spreadsheet letter (A, B, AA, etc.) for formula display
getColLetter(n) → string	2366	Convert 0-based index to Excel-style column letter
addAccount()	3693	Append a blank evaluation row to <code>currentData.evaluations</code> and re-render
deleteEvaluation(index)	3705	Soft-delete evaluation row (set <code>_deleted=true</code>), save, re-render
updatePropProgress(input, index, key)	2428	Handle inline input change on Prop Day column; call <code>handleInputChange</code>
evalSpinnerHTML() → string	2052	Return HTML string for the loading spinner overlay

17.3 Cell Editing & Input Helpers

Function	Line	Description
handleCellClick(event, index, key)	1485	Distinguish note cells from editable cells; route to correct handler
handleInputChange(index, key, value)	3668	Update <code>currentData.evaluations[index][key]</code> and debounce <code>saveData()</code>

Function	Line	Description
getInputHtml(value, type, index, key, readOnly)	3441	Return HTML for a text/number/date/checkbox input cell based on column type
getDropdownHtml(value, type, index, key, readOnly)	3338	Return HTML for a dropdown cell with all options populated
toggleMenu(e, dropdown)	3614	Show/hide a custom dropdown menu; position it relative to the trigger element
selectItem(e, item, index, key)	3639	Handle option selection from custom dropdown; update data + re-render
closeGlobalMenu()	3584	Hide any open custom dropdown menu
updateGlobalMenuPosition()	3595	Recompute dropdown position on scroll/resize
formatCurrencyValue(value) → string	1937	Format a number as a currency string with commas and 2 decimal places

17.4 Inline Notes System

Function	Line	Description
openNoteEditor(event, index, key)	1500	Show the note modal for a cell; populate with existing note text if any
enableNoteEdit()	1633	Switch note modal from read-only to edit mode
closeNoteModal()	1646	Hide the note modal and reset state
saveNote()	1656	POST /api/notes with {client_id, row_index, column_key, content}; hide modal
deleteNote()	1688	POST /api/notes/delete; re-render cell without note indicator

17.5 Scroll & Section Navigation

Function	Line	Description
calculateSectionPositions()	1775	Measure and cache the X-offset of each section group header in the table
scrollTableBy(amount)	1807	Scroll the .table-wrapper div by a relative pixel amount
scrollTableTo(position)	1813	Scroll the .table-wrapper div to an absolute X position
scrollToSection(section)	1822	Scroll to the start X of the named section ('eval-info', 'funded-phase', etc.)
updateScrollIndicator()	1829	Update the floating section pill label based on which group header is centred in view

17.6 Stats, Positions & Accounts

Function	Line	Description
renderStats(data)	2458	Populate the Stats tab with account balance, win rate, drawdown, and phase counts

Function	Line	Description
renderPositions(data)	2668	Render open MT5 positions table in the Positions tab
renderHedgeAccounts(data)	2701	Render hedge account rows (login, balance, deposits, withdrawals, P&L;)
renderPropAccounts(data)	2779	Render prop account rows with inline editing
renderVPSAccounts(data)	2891	Render VPS account rows with inline editing
updateHedgeAccount(index, key, value)	2970	Update a hedge account field in currentData and debounce save
addHedgeAccount()	2977	Append blank hedge account to currentData.hedge_accounts; re-render
deleteHedgeAccount(index)	2991	Remove hedge account at index; re-render; save
updatePropAccount(index, key, value)	2862	Update prop account field in currentData
addPropAccount()	2869	Append blank prop account
deletePropAccount(index)	2884	Remove prop account at index
updateVPSAccount(index, key, value)	2944	Update VPS account field
addVPSAccount()	2951	Append blank VPS account
deleteVPSAccount(index)	2963	Remove VPS account at index

17.7 Trade History Tab

Function	Line	Description
renderHistory(data)	2998	Populate the History tab with the MT5 deals list
parseDealTime(raw) → int	3007	Parse MT5 deal timestamp (Unix int or ISO string) to milliseconds
formatDealTime(raw) → str	3015	Format a deal timestamp as a human-readable string
toggleHistorySort()	3024	Toggle sort direction on the history table and re-render
renderHistoryTable()	3031	Build and insert HTML for the deals history table using current sort order

17.8 Hedging Review Tab

Function	Line	Description
formatMoney(value) → str	3075	Locale-formatted currency string with sign (e.g. '+\$1,234.56')
updateHedgingCalc()	3081	Recompute derived hedging fields (actual result, discrepancy) from input values
saveHedgingReview(retryCount=0)	3099	async — POST /api/hedging_review/ with current hedging section data; retry on 429
renderHistoricalMT5List(accounts)	3151	Render the list of historical (closed) MT5 accounts in the hedging panel

Function	Line	Description
showAddHistoricalMT5Modal()	3189	Show modal form to add a historical MT5 account record
closeHistoricalMT5Modal()	3231	Hide and reset the historical MT5 modal
saveHistoricalMT5(retryCount=0)	3236	async — POST /api/historical_mt5/ to add a new historical account
deleteHistoricalMT5(index, retryCount=0)	3290	async — POST /api/historical_mt5/ with action='delete' and index

17.9 Save, Version History & Rollback

Function	Line	Description
saveData(action='UPDATE', description, retryCount=0)	3734	Debounced 600 ms — POST /api/update_data with full currentData; show status; retry on 429
showSaveStatus(status, detail='')	3812	Display a coloured status badge (Saved ✓ / Saving... / Error) in the header
openHistoryPanel()	3861	Slide the history panel into view; call loadVersionHistory()
closeHistoryPanel()	3867	Hide the history panel
loadVersionHistory()	3872	Fetch /api/client/history and store in historyVersions
renderHistoryList()	3922	Build the version list UI (version number, timestamp, action, user)
selectHistoryVersion(version, idx)	3976	Highlight selected version in list; enable preview/restore buttons
previewSelectedVersion()	3988	Fetch snapshot for selected version and enter preview mode
enterPreviewMode(versionData, versionInfo)	4012	Swap currentData with snapshot data; show orange 'PREVIEW' banner
exitPreviewMode()	4040	Restore original currentData; hide preview banner
restoreFromPreview()	4055	Call restoreVersion() with the currently previewed version
restoreSelectedVersion()	4060	Call restoreVersion() with the selected version from the list
restoreVersion(version)	4070	POST /api/client/rollback; on success reload data and close panel

17.10 Waterlog / Watermark Tab

Function	Line	Description
loadWaterlogData()	4099	Fetch /api/sheet/waterlog and render waterlog period table
loadWatermarkData()	4111	Fetch /api/client/watermark_history/ and render daily watermark chart
formatDateForInput(dateStr) → str	4272	Convert ISO date string to YYYY-MM-DD format for

END OF DOCUMENT

Generated March 01, 2026 · github.com/Oukaharry/FuturesAutomatedFeed